

LINQ Aggregate Operators in C#

In this article, I will discuss the **LINQ Aggregate Operators in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the [LINQ Reverse Method in C#](#) with Examples. As part of this article, we will discuss the following concepts.

1. [What are Aggregate Functions in Linq in C#?](#)
2. [When to use Aggregate Functions?](#)
3. [Types of Aggregate Functions](#)

What are LINQ Aggregate Operators in C#?

LINQ Aggregate Operators perform mathematical operations on sequences of numeric data and are also used to compute a single value from a sequence of values. These operators are used to group together the values of multiple rows as the input and then return the output as a single value. So, in simple words, we can say that the aggregate methods in C# will always return a single value.

Whenever you want to perform some mathematical operations such as Sum, Count, Max, Min, Average, and Aggregate on the numeric property of a collection, then you need to use the LINQ Aggregate Functions.

What are the Aggregate Methods/Operators Provided by LINQ?

LINQ provides aggregate operators that allow you to perform aggregate calculations on elements in a collection or sequence, such as sum, average, minimum, maximum, and more. The following are the aggregate methods provided by Linq to perform mathematical operations on a collection.

1. **Sum():** This method calculates the collection's total(sum) value.
2. **Max():** This method is used to find the largest value in the collection
3. **Min():** This method is used to find the smallest value in the collection
4. **Average():** This method is used to calculate the average value of the numeric type of the collection.
5. **Count():** This method counts the number of elements in the collection.
6. **Aggregate():** This method is used to Perform a custom aggregation operation on the values of a collection.

LINQ Count and LongCount Methods:

Counts the number of elements in a sequence. It can also be used with a predicate to count elements that meet a certain condition. LongCount is used when the count might exceed the range of an int. The Count method returns the number of elements in a collection that satisfy a specified condition.

Use Cases of Count and LongCount:

- When you need to know the number of items in a collection.
- To determine if a collection is empty (by checking if the count is zero).

Syntax: `int count = collection.Count(element => condition);`

Example:

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            var numbers = new List<int> { 1, 2, 3, 4, 5 };
            int countEven = numbers.Count(x => x % 2 == 0); // 2
            Console.WriteLine($"Event Number Count: {countEven}");
            Console.ReadKey();
        }
    }
}
```

LINQ Sum Method:

Calculates the sum of the numeric values in a sequence. For example, if you have a collection of integers, you can use Sum to find their total. The Sum operator calculates the sum of numeric values in a collection.

Use Cases of Sum:

- To calculate the total of a series of numbers.
- To find the sum of a specific numeric property across all objects in a collection.

Syntax: `int sum = collection.Sum(element => element.Value);`

Example:

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            var expenses = new List<double> { 100.50, 75.25, 50.0, 30.75 };
            double totalExpenses = expenses.Sum(); // 256.5
            Console.WriteLine($"Total Expenses: {totalExpenses}");
            Console.ReadKey();
        }
    }
}
```

LINQ Min Method:

Returns the minimum value in a sequence. The Min operator returns the minimum value from a collection of elements.

Use Cases of Min:

- To find the smallest number in a series.
- To determine the minimum value of a particular property in a collection of objects.

Syntax: `T min = collection.Min();`

Example:

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            var temperatures = new List<int> { 10, 5, 15, 0, -5 };
            int minTemperature = temperatures.Min(); // -5
            Console.WriteLine($"Min Temperature: {minTemperature}");
            Console.ReadKey();
        }
    }
}
```

LINQ Max Method:

Returns the maximum value in a sequence. The Max operator returns the maximum value from a collection of elements.

Use Cases of Max:

- To identify the largest number in a sequence.
- To find the maximum value of a specific property within a set of objects.

Syntax: `T max = collection.Max();`

Example:

```
using System;
using System.Collections.Generic;
using System.Linq;
```

```

namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            var prices = new List<decimal> { 25.99m, 15.49m, 30.0m, 10.25m };
            decimal maxPrice = prices.Max(); // 30.0
            Console.WriteLine($"Max Price: {maxPrice}");
            Console.ReadKey();
        }
    }
}

```

LINQ Average Method:

Computes the average of the numeric values in a sequence. It sums up the elements and then divides by the count of elements. The Average operator calculates the average of numeric values in a collection.

Use Cases of Average:

- To calculate the mean of a sequence of numbers.
- To determine the average value of a particular property for a collection of objects.

Syntax: `double average = collection.Average(element => element.Value);`

Example:

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            var scores = new List<double> { 85.5, 92.0, 78.25, 95.75 };
            double averageScore = scores.Average(); // 87.625
            Console.WriteLine($"Average Score: {averageScore}");
            Console.ReadKey();
        }
    }
}

```

LINQ Aggregate Method:

This is the most flexible aggregate operator. It applies a function you define to each element in a sequence, accumulating the results. This operator can be used to create custom aggregations that aren't directly

provided by other aggregate operators.

Use Cases of Aggregate:

- For more complex aggregations that cannot be expressed with the other operators, such as concatenating strings, cumulative products, or running totals.
- When you need to keep track of intermediate results in the aggregation process.

Syntax: `TResult result = collection.Aggregate(seed, (accumulator, element) => aggregationFunction);`

Example:

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            var numbers = new List<int> { 1, 2, 3, 4, 5 };
            int product = numbers.Aggregate((accumulator, number) => accumulator * number); //Return 120 => 1 * 2 * 3 * 4 * 5
            Console.WriteLine($"Aggregate: {product}");
            Console.ReadKey();
        }
    }
}
```

When to Use LINQ Aggregate Operators in C#?

Here are some common scenarios where LINQ aggregate operators are particularly useful:

- **Summarizing Data:** When you need to summarize data, such as finding the sum, average, or count of elements in a collection, aggregate operators are very useful. For example, you can use `Sum` to calculate the total of all numbers in a list or `Count` to determine the number of elements in a collection.
- **Finding Minimum or Maximum Values:** If you need to find the minimum or maximum value in a collection, you can use the `Min` or `Max` operators. These are handy for scenarios like finding a data set's highest or lowest score.
- **Counting Elements:** Use the `Count` operator to determine the number of elements in a collection that meet specific criteria. Example: Counting the number of active users in a user database.
- **Aggregate Calculations:** The `Aggregate` operator is a versatile tool for performing custom aggregate calculations. It can be used for operations that are more complex than what is provided by the standard operators, like calculating the product of all elements in a sequence.
- **Custom Aggregation with Seed Value:** If you have a more complex aggregation requirement where you want to start the aggregation with a specific seed value and then apply an accumulator function, you can use the `Aggregate` method with a seed value.
- **Combining Elements:** Operators like `Aggregate` can also be used to combine elements of a sequence into a single value in a customized way, such as concatenating strings or combining data

structures.

- **Grouping and Summarizing by Key:** Use aggregate operators in conjunction with grouping operators to group data by a key and perform aggregations within each group. Example: Grouping sales data by product category and calculating the total sales for each category.
- **Database Queries:** In LINQ to SQL or LINQ to Entities, aggregate operators to create efficient database queries that return aggregated results from database tables. Example: Retrieving the total revenue from a sales database table.

In the next article, I will discuss the [LINQ Sum Method in C#](#) with Examples. In this article, I explain LINQ Aggregate Operators in C# with Examples, and I hope you enjoy this article.